

Дослідження

Напрямок: Зв'язні списки та інші лінійні структури даних

Базові теми: Циклічний список, Список з sentinel nodes

Цікаві теми: XOR linked list

1. Циклічний список (Circular Linked List)

Що це таке?

Циклічний список — це різновид зв'язаного списку, у якому останній вузол не має вказівника на null (що зазвичай означає кінець списку). Натомість його вказівник next посилається назад на перший вузол (head), утворюючи замкнене коло. Циклічним може бути як однозв'язний (Singly), так і двозв'язний (Doubly) список. [Візуалізація](#)

Особливості та реалізація

Як зазначається в навчальних матеріалах, важливою деталлю оптимізації циклічного списку є зберігання вказівника не на початок (head), а на кінець списку (tail). Оскільки tail->next завжди вказує на початок, ми маємо миттєвий доступ одночасно і до першого, і до останнього елементів. Це дозволяє здійснювати вставку елементів як на початок, так і в кінець списку за константний час $O(1)$ без необхідності обходити весь список.

Практичне застосування

- **Round-Robin планування:** Алгоритми операційних систем часто використовують циклічні списки для диспетчеризації процесів. Процесору передаються задачі по черзі, і коли список доходить до кінця, він автоматично повертається до першого процесу.
- **Багатокористувацькі ігри:** Черга ходів гравців (після останнього гравця хід знову переходить до першого).
- **Відсутність null:** Під час обходу списку не потрібно робити перевірку на обрив, що спрощує логіку певних алгоритмів.

Питання під час дослідження

Як запобігти нескінченному зацикленню під час обходу списку? Замість стандартної перевірки while (current != NULL), необхідно запам'ятати початковий вузол і використовувати умову while (current != head). Чи існують ризики витоку пам'яті (memory leaks) при видаленні циклічного списку? (Питання залишено для подальшого вивчення).

Практична реалізація (C++)

Замість класичного зберігання вказівника на початок (head), структура оптимізована шляхом збереження лише вказівника на кінець (tail). Оскільки список зациклений, вираз `tail->next` завжди вказує на перший елемент. `push_back` (додавання в кінець): при надходженні нового елемента алгоритм виконує такі кроки:

1. Поле `next` нового вузла зв'язується з поточним початком списку (`tail->next`).
2. Поле `next` поточного хвоста перенаправляється на новий вузол.
3. Вказівник `tail` зсувається на новий вузол. Такий підхід забезпечує додавання елементів за стабільний константний час $O(1)$ без необхідності повного обходу структури.

```
struct CNode
{
    int data;
    CNode*next;
};class CircularList
{
    CNode*tail;public:
    CircularList():tail(nullptr){}

    //Вставка в кінець за O(1)
    void push_back(int data) {
        CNode*newNode=newCNode{data,nullptr}; if
        (!tail) {
            tail = newNode;
            tail->next=tail;
        }else{
            newNode->next=tail->next;
            tail->next = newNode;
            tail=newNode;
        }
    }
};
```

2. Список із фіктивними вузлами (Sentinel Nodes)

Що це таке?

Sentinel nodes (фіктивні вузли) — це спеціальні (часто "порожні" або з фіктивним значенням) вузли, які навмисно додаються на початок або в кінець зв'язаного списку для того, щоб спростити граничні умови (edge cases). Вони не містять реальних даних користувача і слугують лише каркасом. [Візуалізація](#)

Як вони працюють і навіщо потрібні?

Зазвичай під час маніпуляцій зі зв'язаним списком розробнику доводиться писати багато умовних операторів (наприклад: `if (head == null)`, щоб перевірити, чи не порожній список). Додавання sentinel вузлів вирішує цю проблему:

- Ви гарантуєте, що ваш список ніколи не буває повністю порожнім.
- Операція додавання або видалення першого реального елемента стає абсолютно ідентичною до операцій всередині списку.

- Зменшується кількість коду, алгоритм працює швидше і виключаються помилки `NullPointerException`.

Практичне застосування

Їх широко використовують у вирішенні алгоритмічних задач (як було показано на прикладі задачі `Partition List` на `LeetCode`). Стандартні бібліотеки багатьох мов програмування (наприклад, `std::list` у `C++`) використовують `sentinel` вузли.

Питання під час дослідження

Якщо `Sentinel nodes` роблять код безпечнішим, чому їх не застосовують абсолютно скрізь? Через витрати пам'яті. Якщо в програмі створюється масив із тисяч коротких списків (наприклад, у хеш-таблиці), використання фіктивних вузлів для кожного з них призведе до значного марнотратства оперативної пам'яті.

Практична реалізація (C++)

У конструкторі класу під час ініціалізації миттєво створюються та зв'язуються два фіктивні (`dummy`) вузли — `header` (голова) та `trailer` (хвіст). `push_back`: Завдяки наявності фіктивних вузлів, цей список системно ніколи не буває порожнім. Додавання нового елемента завжди відбувається строго між останнім існуючим елементом (`trailer->prev`) та фіктивним хвостом (`trailer`). Це архітектурне рішення повністю усуває необхідність використання умовних розгалужень (`if/else`) для перевірки граничних умов (`edge cases`), що оптимізує роботу конвеєра процесора.

```
struct DNode
{
    int data;
    DNode*prev;
    DNode*next;
};classSentinelList{ DNode
    de* header;
    DNode*trailer;public:
    SentinelList(){
        //Ініціалізація вузлів
        header = new DNode{0, nullptr, nullptr};
        trailer=newDNode{0,nullptr,nullptr};
        header->next = trailer;
        trailer->prev=header;
    }

    //Вставка безжодних перевірок на NULL
    void push_back(int data) {
        DNode*newNode=newDNode{data,trailer->prev,trailer};
        trailer->prev->next = newNode;
        trailer->prev=newNode;
    }
};
```

3. XOR Linked List (Оптимізований за пам'яттю список)

Що це таке?

XOR-список — це надзвичайно ефективна щодо використання пам'яті версія двозв'язаного списку. У стандартному списку кожен вузол має два вказівники (prev та next). Кожен вказівник займає місце в пам'яті (8 байт на 64-бітних архітектурах). [Візуалізація](#)

Робота з пам'яттю та операція XOR

Щоб зменшити витрати пам'яті вдвічі, замість двох вказівників XOR-список використовує лише один — `npx` (`next-prev-XOR`). Він не вказує безпосередньо на жоден вузол, а зберігає результат математичної побітової операції виключного АБО: $npx = \text{адреса}(\text{prev}) \text{ XOR } \text{адреса}(\text{next})$. Як здійснюється обхід списку: Цей підхід працює завдяки властивостям операції: $A \text{ XOR } (A \text{ XOR } B) = B$. Якщо ви зараз на вузлі `Current` і знаєте адресу `Previous`, ви можете дізнатися адресу наступного вузла так: $\text{адреса}(\text{Next}) = \text{адреса}(\text{Previous}) \text{ XOR } \text{Current.npx}$. Це дозволяє повноцінно пересуватися вперед і назад по списку.

Особливості та обмеження

- **Переваги:** Суттєва економія пам'яті, що є критично важливим для вбудованих систем (embedded systems) та мікроконтролерів.
- **Обмеження:** Сучасні мови програмування зі збирачами сміття (Java, Python) не дозволяють прямо працювати з адресами. Такий код набагато складніше читати та відлагоджувати (debug).

Питання під час дослідження

Чому компілятор C++ видає помилку під час спроби застосувати оператор $\wedge$ до вказівників? Відповідь: Стандарт C++ забороняє виконувати побітові операції над типами вказівників. Для виконання XOR адреси необхідно "кастувати" до цілочисельного типу `uintptr_t` (бібліотека `<stdint>`).

Практична реалізація (C++)

Використання єдиного поля `npx` для імітації двозв'язності. Оскільки стандарт мови C++ забороняє застосовувати побітові операції безпосередньо до вказівників пам'яті, реалізовано допоміжну функцію XOR. Вона тимчасово приводить вказівники до безпечного цілочисельного типу `uintptr_t` (бібліотека `<stdint>`), виконує операцію $\wedge$ і повертає результат у вигляді адреси. `push_back`: Оскільки новий елемент додається в кінець, його наступним віртуальним вузлом є `nullptr`. Тому його поле обчислюється як $npx = \text{XOR}(\text{tail}, \text{nullptr})$. Найскладнішим етапом є оновлення `npx` попереднього хвоста: алгоритм спочатку "розпаковує" адресу його попередника ($\text{XOR}(\text{tail} \rightarrow \text{npx}, \text{nullptr})$), а потім перезаписує `npx`, зв'язуючи його з новим доданим елементом.

```
#include<cstdint>structXORNode{ in
    t data;
    XORNode*np;
};class XORList
{ XORNode*head;
  XORNode*tail;

  //Функція для виконання операції XOR над адресами
  XORNode* XOR(XORNode* a, XORNode* b) {
      return(XORNode*)((uintptr_t)(a)^(uintptr_t)(b));
  }public:
  XORList():head(nullptr),tail(nullptr){}

  voidpush_back(intdata){
      XORNode*newNode=newXORNode{data,XOR(tail,nullptr)}; if
      (!head) {
          head=tail=newNode;
      }else{
          tail->np=XOR(XOR(tail->np,nullptr),newNode); tail =
          newNode;
      }
  }
};
```

4. Порівняння зі стандартними аналогами

Обов'язковим етапом дослідження є розуміння місця реалізованих структур в екосистемі мови C++.

Власна реалізація	Аналог у C++ (STL / Boost)	Різниця / Особливості застосування
CircularList	Прямого аналога в STL немає. Зазвичай емулюється через std::list або std::vector з модульною арифметикою (%).	Власна структура економить пам'ять на абстракціях та забезпечує гарантований O(1) для переходу від кінця до початку.
SentinelList	std::list (у більшості імплементацій компіляторів GCC/Clang "під капотом" використовується dummy/sentinel-вузол).	Власна реалізація демонструє низькорівневий механізм того, як саме STL уникає помилок на кінцях списку.
XOR List	Немає в STL. Близько стоїть boost::intrusive::slist, але він працює за іншим принципом.	Демонструє радикальну оптимізацію пам'яті (до 33% економії на вказівниках) ціною втрати читабельності та ускладнення дебагінгу.

5. Планування експерименту та Benchmarking

Для комплексного підтвердження працездатності алгоритмів було написано Unit-тести (всі тести успішно пройдені: [PASS]) та проведено експеримент, розділений на два етапи аналізу.
(Усі сирі дані згенеровані кодом автоматично, додані у папку logs.

Етап 1: Аналіз часової складності (Швидкодія)

Методологія: Виміряно час виконання операції `push_back` (додавання в кінець) для поступово зростаючих масивів даних: від 10 000 до 200 000 елементів. Щоб виконати вимогу використання непередбачуваних даних, було задіяно бібліотеку `<random>` — у списки вставлялися випадково згенеровані числа. Як еталон швидкості використано `std::list`.

Точні результати вимірювання часу в мілісекундах:

Кількість елементів	Circular List	Sentinel List	XOR List	std::list (Еталон)
10 000	0.591373 ms	0.630351 ms	0.819013 ms	1.12442 ms
50 000	1.147470 ms	1.551580 ms	1.630330 ms	3.64844 ms
100 000	2.395680 ms	2.648950 ms	3.303950 ms	7.12617 ms
200 000	3.388360 ms	6.932320 ms	12.003800 ms	22.25760 ms

Що доводить цей етап: Власні реалізації списків виявилися у 2-3 рази швидшими за стандартну бібліотеку. Відсутність додаткових перевірок граничних умов (`if/else`) у `Circular List` та `Sentinel List` дозволяє процесору виконувати код максимально лінійно.

Етап 2: Аналіз використання пам'яті

Методологія: Аналітичний розрахунок обсягу виділеної оперативної пам'яті (RAM) під час зберігання масиву з 1 000 000 елементів. Також у код були додані кастомні деструктори `~Class()`, щоб забезпечити повне вивільнення пам'яті після бенчмарків і уникнути `Memory Leaks`.

Результати вимірювання (Пам'ять):

- `Circular List` (1 вказівник): 15.2588 MB
- `XOR List` (1 віртуальний вказівник): 15.2588 MB
- `Sentinel List` (2 вказівники): 22.8882 MB

Що доводить цей етап: Етап математично доводить головну архітектурну перевагу `XOR List`. Він емулює поведінку двозв'язного списку, але споживає рівно стільки ж пам'яті, як базовий однозв'язний список (15.25 MB), економлячи понад 7 Мегабайт порівняно зі звичайним `Sentinel List` (22.88 MB).

Загальний висновок експерименту: Отримані дані демонструють інженерний закон `Space-Time Trade-off` (компроміс між часом та пам'яттю). `XOR List` виконує вставку трохи довше за інші структури (через побітові обчислення $\wedge$), проте ця мікрожертва процесорного часу дозволяє радикально зекономити оперативну пам'ять.

6. Аналіз роботи з ШІ

Під час роботи над дослідженням інструменти ШІ використовувалися для структурування інформації, оформлення звіту, візуалізації, написання коду, бенчмаркінгу. Чат з Gemini: Посилання

Виявлені помилки ШІ: Під час первинного запиту щодо реалізації XOR-списку, ШІ (Gemini) згенерував синтаксично некоректний код, намагаючись виконати операцію `tail->npx ^ newNode` безпосередньо над

типами вказівників XORNode*. Ця помилка була виявлена під час компіляції.

Виправлення: Логіку було скориговано шляхом примусового кастування адрес до цілочисельного типу `uintptr_t` з бібліотеки `<stdint>` всередині кастомної функції XOR(). Також у початкових кодах III були відсутні деструктори, що призводило до Memory Leaks під час повторних бенчмарків. У фінальну версію було додано деструктори з деаллокацією пам'яті через `delete`.

7. Історичні відомості, практичні застосування, обговорення на форумах

Про Circular List (Алгоритм Round-Robin)

Циклічні списки є основою для алгоритму Round-Robin (кругове обслуговування), який масово використовується в індустрії:

- Планування в операційних системах (OS Scheduling): Як зазначається в технічних матеріалах Scaler, сучасні ОС використовують циклічні черги для розподілу процесорного часу. Процесор виділяє кожній задачі рівний квант часу (time slice), а після його завершення процес автоматично переміщується в кінець циклічного списку, поступаючись місцем наступному.
- Балансування мережевого навантаження (Load Balancing): За даними CDNetworks, балансувальники трафіку використовують цей патерн для рівномірного розподілу запитів від користувачів між кількома серверами у дата-центрах. Запити передаються серверам строго по колу, що запобігає перевантаженню будь-якого з них.
- Автоматизація бізнес-процесів (Task Routing): У CRM-системах, службах підтримки та розумних календарях (наприклад, Cal.com та Reclaim.ai) циклічні списки забезпечують справедливий розподіл навантаження. Нові заявки або дзвінки призначаються вільним менеджерам по черзі, замикаючи коло після останнього працівника.
- Турнірні сітки (Tournament Logic): У кіберспорті та спортивних застосунках (як Tournamently) кругова система (кожен грає з кожним) програмно найпростіше реалізується саме через зациклені масиви даних, де вказівник послідовно зміщується для формування пар супротивників.

[Scaler Topics: Round-Robin Scheduling in OS](#)

[CDNetworks Glossary: What is Round Robin?](#)

[Cal.com Wiki: Round-Robin Scheduling: Use Cases & Examples Across Industries](#)

[Reclaim.ai Blog: What is a Round Robin?](#)

[Tournamently Blog: Round Robin Tournament Format Explained](#)

Про Sentinel Nodes

Було знайдено різні обговорення про доцільність використання `std::list` у C++ та архітектуру Linux:

- Реалізація `std::list` у C++ (Sentinel Nodes): Як розбирається в навчальних матеріалах CodeSignal, контейнер `std::list` є класичним двозв'язним списком. Проте його ключова архітектурна особливість — використання Sentinel Node для репрезентації ітератора `end()`. Це дозволяє безпечно виконувати операції вставки/видалення без ризику розіменування нульового вказівника (`NullPointerException`) та зберігає цілісність списку навіть тоді, коли він стає порожнім.
- Критика в індустрії: У реальних проєктах `std::list` використовується обережно. Через фрагментацію пам'яті (кожен вузол виділяється окремо) процесор витрачає багато часу на завантаження даних у кеш (Cache Misses). Тому масиви (`std::vector`) часто працюють швидше, незважаючи на $O(n)$ зсуви. `std::list`

обирають лише тоді, коли критично важливо уникнути інвалідації ітераторів (Iterator Invalidation) при модифікації масиву.

- Ядро Linux та `struct list_head` (Інтрузивні списки): Як описується в аналітиці від Medium та Dev.to, ядро операційної системи Linux (Linux Kernel) використовує власну, унікальну імплементацію двозв'язного циклічного списку. Замість того, щоб список містив дані (як у класичному підході), дані містять список. Структура `list_head` має лише два вказівники (`next` та `prev`) і жодних корисних даних. Вона вбудовується безпосередньо всередину інших структур ядра (наприклад, процесів або драйверів). Щоб отримати самі дані, розробники ядра використовують геніальний C-макрос `container_of()`, який за допомогою адресної арифметики вираховує початок батьківської структури. Це дозволяє ядру Linux об'єднувати абсолютно різні об'єкти в одну чергу без виділення додаткової пам'яті під вузли списку, що робить алгоритм екстремально швидким і економним.

[CodeSignal: Introduction to std::list in C++](#)

[Дискусія на Reddit \(r/cpp\): Does anybody use std::list?](#)

[Medium \(Boutnaru\): The Linux Kernel Data Structures Journey: struct list_head](#)

[Dev.to: Linked List in the Linux Kernel](#)

Про XOR linked list

Було знайдено статті про життєвий цикл цієї структури даних:

- Епоха дефіциту пам'яті: Структура набула популярності у 1980-х роках. У той час обсяг оперативної пам'яті (RAM) комп'ютерів вимірювався кілобайтами. Економія 4 або 8 байтів на кожному вузлі двозв'язного списку була не просто оптимізацією, а питанням життєздатності програми. XOR-список вважався геніальним хаком, що дозволяв отримати функціонал двозв'язного списку за ціною однозв'язного.
- Чому XOR-списки майже не використовують у сучасному Enterprise-програмуванні: Згідно з дискусіями системних інженерів, сьогодні ця структура має три критичні недоліки, через які вона стала радше академічним концептом:
 - Несумісність із Garbage Collection (Збиранням сміття): Сучасні мови програмування (Java, C#, Go, Python) використовують автоматичне очищення пам'яті. Збирач сміття (GC) сканує пам'ять у пошуках дійсних вказівників. Оскільки в прх лежить "сміття" (результат побітової операції $\wedge$), GC не розпізнає його як адресу і може випадково видалити вузли списку.
 - Дебагінг: При виникненні помилки розробник не може просто відкрити відлагоджувач (наприклад, GDB) і подивитися, куди вказує наступний вузол. Адресу потрібно розраховувати вручну, що робить підтримку коду надзвичайно дорогою.
 - Кеш-промахи: Сучасні процесори працюють дуже швидко, але пам'ять — повільно. Тому процесор намагається завантажити дані наперед. У випадку з XOR-списком процесор не може передбачити наступну адресу, поки не виконає математичну операцію, що повністю руйнує апаратну оптимізацію.
- Спадщина в Embedded Systems (Вбудованих системах): Попри те, що сам XOR-список використовується рідко, принципи, на яких він побудований — жорстка маніпуляція пам'яттю та побітові операції — є абсолютною базою для розробки Embedded-систем (прошивки для мікроконтролерів, IoT-пристроїв, автомобільної електроніки). У цих сферах ресурси досі обмежені, і вміння працювати на рівні бітів та адрес (через типи на кшталт `uintptr_t`) є критично важливою навичкою для Embedded C/C++ інженера.

[Grokopedia: XOR Linked List Structure](#)

[DevGenius: XOR Doubly Linked List Implementation](#)

[Hacker News \(YCombinator\): Інженерна дискусія щодо доцільності XOR Linked List](#)
[Medium \(Alwin T3r\): Core Embedded Systems Skill: Bitwise Operations](#)

8. Офіційні джерела та матеріали дослідження

Циклічний список (Circular Linked List):

- [GeeksforGeeks: Circular Linked List in C++](#)
- [GeeksforGeeks: DSA Circular Linked List \(Теорія та алгоритми\)](#)
- [Reddit \(r/learnprogramming\): Practical application of Circular Linked Lists](#)
- [YouTube \(Навчальний гайд\): Circular Linked List Tutorial - Why Use a Circular List?](#)
- [YouTube \(Візуалізація алгоритму\): Circular Linked List Implementation](#)

Список із Sentinel Nodes:

- [GeeksforGeeks: Doubly Linked List Using Sentinel Nodes](#)
- [Medium: Linked List Pattern: Sentinel Node](#)
- [Reddit \(r/cs2a\): Understanding sentinel nodes and their importance](#)
- [YouTube \(Теоретичний розбір\): What are Sentinel Nodes?](#)

XOR Linked List:

- [GeeksforGeeks: XOR Linked List – A Memory Efficient Doubly Linked List](#)
- [YouTube \(Математичне обґрунтування та алгоритм обходу\): XOR Linked List Explanation](#)

Управління задачами в рамках дослідження здійснювалося за методологією Kanban:

Live-дошка: [Посилання на Trello-дошку проєкту](#)